

অপারেটিং সিস্টেমের ভূমিকা

(Introduction to Operating Systems)

প্রফেসর চেস্টার রেবিয়ার

(Prof. Chester Rebeiro)

কম্পিউটার সায়েন্স এন্ড ইঞ্জিনিয়ারিং

(Department of Computer Science and Engineering)

ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি (Indian Institute of Technology), মাদ্রাজ

সপ্তাহ – 01 বক্তৃতা- 16

সফটওয়্যার ইন্টারপ্ৰেটস এবং সিস্টেম কল

এই ভিডিওটিতে আমরা সফটওয়্যার ইন্টারপ্ৰেট নামে পরিচিত একটি গুরুত্বপূর্ণ ধরনের ইন্টারপ্ৰেট এবং তাদের সিস্টেম কলগুলিতে তাদের অ্যাপ্লিকেশন দেখবো।

(স্লাইডটাইপ পড়ুন: 00:28)

পূর্বের ভিডিওতে, আমরা হার্ডওয়্যার ইন্টারপ্ৰেট দেখেছিলাম। আমরা দেখেছি কিভাবে কী-বোর্ড বা নেটওয়ার্ক কার্ডের মতো একটি ডিভাইস সিপিইউতে একটি নির্দিষ্ট সিগন্যাল জারি করতে পারে এবং এটিকে ডিভাইসের সাথে সংশ্লিষ্ট ইন্টারপ্ৰেট হ্যান্ডলারকে অনিয়ত ভাবে এক্সিকিউট করে। অতএব, আমরা আগের ভিডিওগুলিতে দেখেছি, এই ডিভাইসটি সাধারণত কোনও PIC বা একটি প্রোগ্রামেবল ইন্টারপ্ৰেট নিয়ন্ত্রক হিসাবে একটি মধ্যবর্তী ডিভাইসের মাধ্যমে CPU- এ একটি সংকেত প্রেরণ করবে। অনেকটা একইভাবে, আমাদের কাছে একটি সফটওয়্যার ইন্টারপ্ৰেট পরিচিত। যাইহোক, একটি বহিরাগত ডিভাইস যার ফলে ইন্টারপ্ৰেট, এখানে প্রোগ্রামের একটি ইন্সট্রাকশন ইন্টারপ্ৰেটকে ড্রিগার করবে। এই বিশেষ ক্ষেত্রে, উদাহরণস্বরূপ, যেমন INT হিসাবে একটি ইন্সট্রাকশন ইন্টারপ্ৰেট ঘটবে এবং অপারেটিং সিস্টেম এক্সিকিউট হবে। সুতরাং, এখানে নির্দেশটি INT x হয়, তাই x এখানে ইন্টারপ্ৰেট সংখ্যা। এটি সাধারণত 256 এর কম মানের হয় এবং এটি সফটওয়্যার ইন্টারপ্ৰেটগুলির মধ্যে নির্দিষ্ট বা নির্দিষ্ট করার জন্য ব্যবহৃত হয়।

(স্লাইডটাইপ পড়ুন: 01:54)

সূতরাং, সফটওয়্যার ইন্টারাপ্ট কোথায় ব্যবহার করা হয়? সূতরাং, সফটওয়্যার ইন্টারাপ্ট সিস্টেম কল বাস্তবায়ন করতে ব্যবহৃত হয়। সূতরাং, যেহেতু আমরা জানি একটি ব্যবহারকারীর প্রসেস কিছু কার্নেল অপারেশন সম্পাদনের জন্য একটি সিস্টেম কল চালু করতে পারে। উদাহরণস্বরূপ, এটি একটি ফাইল পড়তে বা একটি ফাইল লিখতে, একটি মনিটর থেকে কিছু মুদ্রণ, অথবা নেটওয়ার্কে একটি প্যাকেট প্রেরণ করতে এবং আরও অনেক কিছু হতে পারে। আরো বিশেষভাবে, সমস্ত অপারেটিং সিস্টেম একটি নির্দিষ্ট সফটওয়্যার ইন্টারাপ্টের মাধ্যমে সিস্টেম কল বাস্তবায়ন করে। উদাহরণস্বরূপ, লিনাক্স অপারেটিং সিস্টেমে, সফটওয়্যার ইন্টারাপ্ট 128 হল সিস্টেম কলগুলি নির্দিষ্ট করার জন্য ব্যবহৃত হয়। অতএব, লিনাক্স অপারেটিং সিস্টেমে, যদি আমাদের একটি INT 128 থাকে যা ব্যবহারকারী প্রসেসে এক্সিকিউট হয়, তবে এটি এমন একটি ইন্টারাপ্টের দিকে পরিচালিত করবে যা কার্নেল বা অপারেটিং সিস্টেম এক্সিকিউট করে এবং এর পরে OSটি ইন্টারাপ্টের উপর নির্ভর করে কোড এক্সিকিউট করে। xv6 তে, সফটওয়্যারটি সিস্টেম কলগুলি বাস্তবায়নের জন্য ব্যবহার করা ইন্টারাপ্ট 64 বা ইউনিক প্রসেস INT 64 এর মত ইন্সট্রাকসনটি একটি সিস্টেম কল বাস্তবায়ন করবে।

(স্লাইডটাইপ পড়ুন: 03:19)

সূতরাং একটি উদাহরণ নিতে, আসুন আমরা বিবেচনা করি যে আমাদের অ্যাপ্লিকেশনে এটির printf স্টেটমেন্ট রয়েছে। সূতরাং, printf এই স্ট্রিংটি একটি স্ট্যান্ডার্ড আউটপুটে মুদ্রণ করবে, যা সাধারণত মনিটরের মতো। এখন printf লাইব্রেরির libc এ একটি ফাংশন হিসাবে উপস্থিত থাকে এবং এর ফলে libc ফাংশনটি চালু হবে। এখন libc ফাংশনে স্পেসিফিকেশন STDOUT এর সাথে লেখা সিস্টেম কলটিতে একটি কল আছে। সূতরাং, এখানে STDOUT ফাইল বর্ণনাকারী; এবং এটি একটি বিশেষ ফাইল বর্ণনাকারী যা স্ট্যান্ডার্ড আউটপুট বা মনিটর বোঝানো হয়। সূতরাং লিখিত ফাংশনে, এটি লিনাক্সে xv6 বা INT 128 এ INT 64 চালু করবে এবং একটি সফটওয়্যার ইন্টারাপ্ট হতে পারে। সূতরাং, আমরা জানি যে সফটওয়্যারের ইন্টারাপ্টটি ঘটবে তা ব্যবহারকারীর স্থান থেকে কার্নেল স্পেসে রূপান্তর ঘটবে এবং এর ফলে অপারেটিং সিস্টেম এক্সিকিউট হবে। OS টি তখন নির্ধারণ করবে যে সিস্টেমের কল করার দরুন আসলে ইন্টারাপ্ট ছিল এবং তারপর এটি নির্ধারণ করবে যে সিস্টেমটি কোনটি থেকে এসেছে; এই ক্ষেত্রে, এটি একটি লিখিত সিস্টেম কল থেকে এসেছে এবং এটি STDOUT - স্ট্যান্ডার্ড আউটপুটে লেখা ছিল। তাই অপারেটিং সিস্টেমটি তখন লিখিত সিস্টেম কলটির জন্য হ্যান্ডলার আহ্বান করবে এবং এই হ্যান্ডলার বিভিন্ন ডিভাইসের সাথে যোগাযোগের যন্ত্র নেবে যেমন ভিডিও কার্ডের মনিটরে স্ট্রিং প্রদর্শন করা। সূতরাং, এই হ্যান্ডলারটি কার্যকর করার পরে, IRET ইন্সট্রাকসন কার্যকর করা হয় যা কার্নেল স্থান থেকে ইউজার স্পেসে রূপান্তরিত হবে এবং প্রোগ্রামটির এক্সিকিউশন অব্যাহত থাকবে।

(স্লাইডটাইপ পড়ুন: 05:31)

এখন, সাধারণত অপারেটিং সিস্টেম বিভিন্ন ধরনের সিস্টেম কলগুলি সমর্থন করে। সুতরাং, এখানে এই বিশেষ টেবিলটি xv6 দ্বারা সমর্থিত বিভিন্ন সিস্টেম কলগুলি দেখায়। সুতরাং, আগের ভিডিওতে আমরা তাদের কিছু দেখেছি। উদাহরণস্বরূপ, আমরা ফর্ক, exit, wait ইত্যাদি দেখেছি। এবং আপনি বিভিন্ন ধরনের সিস্টেম কল যেমন খোলা, পড়া, লেখা, বন্ধ করা, পরিবর্তন ডিরেক্টরি, ডিরেক্টরি তৈরি করা এবং আরও অনেক কিছু নিয়েও পরিচিত। সুতরাং, এই সিস্টেম কলগুলির প্রতিটি একটি সফ্টওয়্যার ইন্টারফেস যেমন INT 64 হিসাবে এক্সিকিউট করা হবে, কারণ এটি xv6, তাই এটি 64। সুতরাং, প্রতিটি সময় এই সিস্টেম কল একটি ব্যবহারকারী প্রসেস দ্বারা ডাকা হয়, এটি ট্রিগার করবে অপারেটিং সিস্টেমকে এক্সিকিউট করার জন্য। এখন, ওএসের দৃষ্টিকোণ থেকে পরবর্তী প্রশ্নটি জিজ্ঞাসা করবে যে, কীভাবে অপারেটিং সিস্টেম বিভিন্ন সিস্টেম কলগুলির মধ্যে পার্থক্য করে? তাই, আমরা উল্লেখ করেছি যে সমস্ত সিস্টেম কলগুলি xv6 এর জন্য INT 128 এবং লিনাক্সের জন্য INT 64 ব্যবহার করবে। সুতরাং কীভাবে ওএস নির্ধারণ করে যে সিস্টেম কলটি ফর্কিং, wait, sleep, exit, ইত্যাদি বিষয়ে ছিল। মূলত এই পার্থক্যটি ব্যবহারকারী প্রসেস থেকেই আসে।

(স্লাইডটাইপ পড়ুন: 07:09)

যা ঘটে তা হল, INT 64 ইন্সট্রাকশনের আগে, ব্যবহারকারীর প্রসেস একটি সিস্টেম কল নম্বর eax রেজিস্টারে নিয়ে যাবে। সুতরাং, উদাহরণস্বরূপ, এই ইন্সট্রাকশন `MOVE x, eax` সিস্টেম কল নম্বর eax রেজিস্টারে স্থানান্তর করা হবে। এখন অপারেটিং সিস্টেমে প্রতিটি সিস্টেম কল একটি অনন্য সংখ্যা, সিস্টেম কল নম্বর থাকবে। এখন আই.এন.টি ইন্সট্রাকশন দ্বারা পরিচালিত অপারেটিং সিস্টেমটি eax রেজিস্টারটি দেখতে পাবে এবং তারপর সেটিকে নির্ধারণ করবে যে সিস্টেম কলটি কীভাবে চালু করা হয়েছিল। উদাহরণস্বরূপ, xv6 এ, যদি আমরা এই বিশেষ হেডার ফাইলগুলিকে দেখি, আমরা দেখবো বিভিন্ন সিস্টেম কল নম্বর সংজ্ঞায়িত করা আছে। উদাহরণস্বরূপ, এখানে, আমরা প্রতিটি সিস্টেম কলে একটি নির্দিষ্ট নম্বর দেওয়া আছে, উদাহরণস্বরূপ SYS আন্ডারস্কোর (underscore) ফর্ককে দেওয়া হয়েছে 1, SYS underscore exit কে 2 প্রদান করা হয় ইত্যাদি। এখন যখন INT 64 ইন্সট্রাকশন কার্যকর হওয়ার কারণে অপারেটিং সিস্টেম এক্সিকিউট হয়, তখন এই সিস্টেম কল নম্বরগুলি ব্যবহার করে ওএস সিস্টেমের কলটি নির্ধারণ করবে এবং তারপর সংশ্লিষ্ট সিস্টেম কল হ্যান্ডলারটি চালু করবে। সুতরাং, প্রতিটি সিস্টেম কলে একটি সংশ্লিষ্ট সিস্টেম কল হ্যান্ডলার আছে। এই এখানে দেখানো হয়েছে, প্রতিটি সিস্টেম কল নম্বর SYS underscore fork যেটা 1, sys underscore exit এটি 2 ইত্যাদি। সুতরাং, এই সিস্টেম কল ফাংশন অপারেটিং সিস্টেমে উপস্থিত যা সিস্টেম কল ধরনের উপর ভিত্তি করে ট্রিগার হয়। উদাহরণস্বরূপ, যদি eax এর মূল্য 11 হয়, তাহলে অপারেটিং সিস্টেম eax রেজিস্টারে দেখবে এবং এটি নির্ধারণ করবে যে এটি getpid সিস্টেম কল আহরণ করে। এবং তারপর এটি এই বিশেষ টেবিলের দিকে

তাকিয়ে দেখবে যে getpid সিস্টেম কলটি এই ফাংশন sys underscore getpid দ্বারা পরিচালিত হয়, এবং সেইজন্য এটি এই sys underscore getpid ফাংশনটি চালু করবে।

(স্লাইডটাইপ পড়ুন: 09:36)

এখন, আসুন আমরা একটি সিস্টেম কল এর সাধারণ প্রোটোটাইপটি দেখি। সুতরাং, এখানে একটি সাধারণ সিস্টেম কল হিসাবে দেখানো হয়েছে। সুতরাং, অবশ্যই, এটির একটি সিস্টেম কলের নাম আছে যা একটি ফাংশন নাম এবং তারপর এটি কিছু রিসোর্স এর বিবরণী এবং প্যারামিটার পাস করে এবং সাধারণত একটি পূর্ণসংখ্যা ফেরত হবে। সুতরাং, রিসোর্স বর্ণনাকারী এখানে নির্দেশ করে যে কোন অপারেটিং সিস্টেমের উৎসটি এখানে লক্ষ্যমাত্রা। উদাহরণস্বরূপ, এটি একটি ফাইল বা একটি ডিভাইস হতে পারে; এবং আমরা আগের স্লাইডে দেখেছি যে এটি একটি নির্দিষ্ট মনিটরকে উল্লেখ করতে পারে। উদাহরণস্বরূপ, যদি রিসোর্স বর্ণনাকারীটি STDOUT হয় তবে এখানে ব্যবহারের জন্য রিসোর্সটি মনিটর। সুতরাং, কিছু সিস্টেম কল এই রিসোর্স বর্ণনাকারীকে নির্দিষ্ট করে না; এই ক্ষেত্রে, সিস্টেম কলটি সেই রিসোর্স এর জন্যই বোঝায়। উদাহরণস্বরূপ, যদি আমরা sleep সিস্টেম কল ব্যবহার করি, আমরা কেবল সময় উল্লেখ করি এবং এমন কোন নির্দিষ্ট ডিস্ক্রিপ্টর যেমন ফাইল, একটি ডিভাইস ইত্যাদি নির্দিষ্ট করি না। এর মানে হল যে বর্তমান প্রসেসে প্রদত্ত ব্যবধানের সময় ঘুমাতে চায়। পরের অংশ হল প্যারামিটার। সুতরাং, এই প্যারামিটারগুলো সিস্টেম কলের জন্য নির্দিষ্ট। উদাহরণস্বরূপ, যদি আমরা পড়তে, লিখতে, খোলা বা বন্ধ বা অন্য কোনও সিস্টেম কল আহ্বান করি, এখানে নির্দিষ্ট করা প্যারামিটারগুলি প্রতিটি সিস্টেম কলগুলির জন্য খুব নির্দিষ্ট হবে। উদাহরণস্বরূপ, লিখিত সিস্টেম কলটিতে প্যারামিটার buf রয়েছে যা একটি ভয়েড পয়েন্টার এবং count। সুতরাং, খোলা বা বন্ধ বা অন্য কোন সিস্টেম কলগুলির বিভিন্ন প্যারামিটার সেট আছে। সুতরাং, মূলত এই প্যারামিটার সিস্টেম কল ধরনের খুব নির্দিষ্ট। রিটার্ন টাইপ সাধারণত int বা পূর্ণসংখ্যা হয়, এবং কখনও কখনও এটি ভয়েড হয়। সুতরাং, 'int' সাধারণত ব্যবহৃত হয় কারণ এই পদ্ধতিতে অপারেটিং সিস্টেম সিস্টেম কল এর সমাপ্তি অবস্থা পাঠাতে সক্ষম হবে, এটা সফলভাবে সম্পাদিত হয়েছে বা এটি ব্যর্থ হয়েছে এবং আরও। কখনো কখনো সিস্টেম কল সম্পর্কে নির্দিষ্ট তথ্য উল্লেখ করার জন্য রিটার্ন ব্যবহার করা হয়। উদাহরণস্বরূপ, রিটার্নটি লিখুন সাইজ আন্ডারস্কোর টি যা প্রকৃতপক্ষে, পূর্ণসংখ্যায় টাইপ করা হয় এবং এটি ফাইলটিতে লেখা বাইটের সংখ্যা নির্দিষ্ট করে, উল্লেখ করে int fd তে। সুতরাং, রিটার্ন টাইপ সিস্টেম কল ধরনের উপর নির্ভর করে পরিবর্তিত হতে পারে। পরের জিনিস যা আমরা দেখব যে কিভাবে এই প্যারামিটারটি রিসোর্স বর্ণনাকারী এবং সিস্টেম কল থেকে প্রাপ্ত প্যারামিটার কার্নেলে পাঠানো হয়। সুতরাং, লক্ষ্য করুন যে সিস্টেম কল একটি পৃথক ফাংশন কল থেকে খুব ভিন্নভাবে প্রয়োগ করা হয়। সুতরাং, একটি ফাংশন কল হিসাবে, আমরা জানি যে ইন্ট্রাকসন কল ব্যবহার করা হবে এবং কলটি একটি ঠিকানা উল্লেখ করবে যেখানে ফাংশন থাকবে এবং কলের জন্য বিভিন্ন প্যারামিটার স্থানীয় স্ট্যাকের মাধ্যমে পাস করা হয়। অনুরূপভাবে, ফাংশন কল থেকে ফেরত

যা int রিটার্নটি eax রেজিস্টারের মাধ্যমে ফিরে আসবে। সুতরাং, ফাংশন কলগুলি থেকে সিস্টেমটি অন্যান্য কাজগুলি আলাদাভাবে কল করে। সুতরাং, যেমন আমরা সিস্টেম কলগুলি দেখেছি, xv6 এর ক্ষেত্রে INT 64 ইন্সট্রাকসন দ্বারা কার্নেলকে আমন্ত্রণ জানানো হয়েছে। সুতরাং, কীভাবে প্যারামিটারগুলি যেমন রিসোর্স ডেসক্রিপ্টর এবং অন্যান্য প্যারামিটারগুলি কার্নেলকে প্রেরণ করে?

(স্লাইডটাইপ পড়ুন: 13:49)

তাই মূলত তিনটি উপায় আছে এটা করার জন্য। প্রথমটি রেজিস্টার দ্বারা পাস করা হয় যা সাধারণত লিনাক্সে করা হয়; দ্বিতীয় উপায় হল ইউজার মোড স্ট্যাকের মাধ্যমে যা xv6 এ করা হয়; এবং তৃতীয় উপায় একটি মনোনীত মেমরি অঞ্চলের মাধ্যমে হয়। সুতরাং, এই বিশেষ ক্ষেত্রে, কি করা হয় যে ব্যবহারকারীর প্রসেস নিজেই, একটি নির্দিষ্ট অঞ্চলের সবচেয়ে বেশি হিপের মধ্যে ব্যবহার করা হবে বিভিন্ন প্যারামিটার সংরক্ষণ করার জন্য যা সিস্টেম কলে প্রেরণ করা প্রয়োজন; এবং হিপ এই অঞ্চলের ঠিকানা রেজিস্টারের মাধ্যমে পাস করা হয়। সুতরাং, আমরা অন্য দুটি ক্ষেত্র আরো বিস্তারিতভাবে দেখবো যেটি রেজিস্টার দ্বারা পাস হয় এবং ইউজার মোড স্ট্যাকের মাধ্যমে পাস হয়।

(স্লাইডটাইপ দেখুন: 14:44)

লিনাক্স সিস্টেম কলগুলি দ্বারা ব্যবহৃত রেজিস্টার দ্বারা পাস করলে কার্নেলের প্যারামিটারগুলি প্রেরণ করতে প্রসেসরে উপস্থিত রেজিস্টারগুলি ব্যবহার করবে। তাই, আমরা জানি আমরা ইতিমধ্যেই এই উদাহরণটি দেখেছি, কিভাবে eax রেজিস্টারটি ব্যবহারকারীর প্রসেসকে অপারেটিং সিস্টেম থেকে সিস্টেম কল নম্বরটি পাস করতে ব্যবহার করা হয়। অনুরূপভাবে, ebx, ecx, esi, edi এবং ebp এর মতো অন্যান্য রেজিস্টারগুলি ইউজার প্রসেস থেকে কার্নেল পর্যন্ত সিস্টেম কলের বিভিন্ন প্যারামিটার পাস করতে ব্যবহার করা হয়। যদি সিস্টেমে 6 টির বেশি আর্গুমেন্ট থাকে, তাহলে একটি ব্লক কার্ঠামোর তালিকা তৈরির পয়েন্টারটি একটি আরগুমেন্ট হিসাবে কার্নেলে প্রেরণ করা হয়।

(স্লাইডটাইপ পড়ুন: 15:41)

এখন, আসুন আমরা দ্বিতীয় কেসটি দেখি যা ইউজার মোড স্ট্যাকের মাধ্যমে পাস হয় এবং এইটি xv6 তে কাজ করে। সুতরাং, এই বিশেষভাবে, int 64 ইন্সট্রাকসন সিস্টেম কলে উপস্থিত বিভিন্ন প্যারামিটার স্ট্যাকের আগে পুশ হবে। উদাহরণস্বরূপ, যদি সিস্টেম কলটির 3 টি প্যারামিটার থাকে; param 1, param 2, এবং param 3, তবে এই তিনটি প্যারামিটার ইউজার স্পেস স্ট্যাকে ঢুকিয়ে দেওয়া হয় এবং তারপর আমরা দেখেছি যে সিস্টেম কল নম্বরটি eax রেজিস্টারে স্থানান্তরিত হয়। সুতরাং, এটি এখানে তিনটি প্যারামিটার ধারণকারী ইউজার প্রসেসে ব্যবহারকারীর স্থান স্ট্যাক। এখন, যখন INT ইন্সট্রাকসন কার্যকর করা হয়, যেমন আমরা জানি, এটি

একটি ইন্টারাপ্ট ট্রিগার করে যা ব্যবহারকারীর স্থান থেকে কার্নেল স্পেসে সুইচ করে দেয়। এছাড়াও এই অন্তর্বর্তীকালীন এক্সিকিউশনের ফলে, যেমন আমরা দেখেছি, স্ট্যাকের মধ্যে ব্যবহারকারী স্থান স্ট্যাক থেকে কার্নেল স্পেস স্ট্যাকে সুইচ হয়। এবং আমরা পূর্বের ভিডিওতে দেখেছি যে এই কার্নেল স্ট্যাকটি ট্র্যাপফ্রেম তৈরি করতে ব্যবহৃত হয়। সুতরাং, এই trapframe এখানে দেখানো হয়েছে। তাই, পূর্ববর্তী ভিডিওতে আমরা যা দেখেছি তা হল ফ্র্যামে এন্ড্রিগুলি সিপিইউ দ্বারা স্ট্যাকে স্বয়ংক্রিয়ভাবে পুশ হয়। সুতরাং, বিশেষ করে, বড় অক্ষরে উল্লিখিত এই রেজিস্টারগুলি সমস্ত কার্নেল স্ট্যাকের উপর চাপিত হয় যা CPU দ্বারা trapframe এ থাকে। এখন, SS এবং ESP এখানে আমাদের জন্য গুরুত্বপূর্ণ। সুতরাং, এই স্ট্যাক সেগমেন্ট এবং স্ট্যাক পয়েন্টার, এবং এই রেজিস্টার ব্যবহারকারী স্পেস স্ট্যাকের অনুরূপ। সুতরাং, যেহেতু আমরা জানি যে INT 64 এক্সিকিউট করা হয়েছে, শেষ পরিচিত স্ট্যাক পয়েন্টারটি এই নির্দিষ্ট অবস্থানে নির্দেশ করছে। এবং তাই, ESP এর বিষয়বস্তু ব্যবহারকারী স্থান স্ট্যাকে এই অবস্থান নির্দেশ করবে। সুতরাং, এই পদ্ধতিতে, কার্নেল তারপর সিস্টেম কলের জন্য বিভিন্ন প্যারামিটার নির্ধারণের জন্য trapframe থেকে SS এবং ESP ব্যবহার করবে।

(স্লাইডটাইপ পড়ুন: 18:২২)

পরের জিনিসটি আমরা দেখব কিভাবে সিস্টেমের কল থেকে ইউজার প্রসেসে ফেরত পাঠানো হয়। আবার আমরা মনে করি যে কার্নেল স্ট্যাকের এই trapframe তৈরি করার সম্পূর্ণ কারণটি এই কারণে হয় যে যখন ইন্টারাপ্ট বা সিস্টেম কলটি কার্যকর করা হয় তখন ট্র্যাপফ্রেমের সম্পূর্ণ অবস্থাটি সংশ্লিষ্ট CPU-র রেজিস্টারের মধ্যে পুনরুদ্ধার করা হয় , এবং এটি ইউজার প্রসেস চালু করে যেখানে এটি বন্ধ করা হয়েছিল সেখান থেকে এক্সিকিউশন অব্যাহত থাকবে এবং ব্যবহারকারীর প্রসেসের কন্ট্রোলারের trapframe এর সাহায্যে পুনরুদ্ধার করা হবে। এখন, সিস্টেম কল থেকে একটি মান ফেরত পাঠাতে যা কিনা কার্নেলে এক্সিকিউট করে তা কার্নেলের স্থানটি থেকে ইউজার স্পেসে ফিরিয়ে আনা হয়, তাই যেটা করা হয় তা হল trapframe-এ eax রেজিস্টার সংশোধন করা হয়; মূলত, আমরা দেখেছি যে এই বিশেষ নির্দেশের কারণে eax রেজিস্টার সিস্টেম কল নম্বর ধারণ করবে। এখন এই সিস্টেম কল নম্বর সিস্টেম কল এর রিটার্ন মান দ্বারা ওভাররাইডেন (overridden) হয়। সুতরাং, এটি একটি নেতিবাচক নম্বর হতে পারে যেমন -1 বা একটি ইতিবাচক সংখ্যা যা আমরা আগের স্লাইডে দেখেছি। তাই, এখন যখন সিস্টেম কল কার্যকর করে এবং তার কার্যকারিতা সম্পূর্ণ করে এবং কন্ট্রোলটি ইউজার প্রসেসে স্থানান্তরিত হয়, তখন ট্র্যাপফ্রেমে eax নতুন মানসহ পুরো ট্র্যাপফ্রেমটি CPU-র রেজিস্টারদের কাছে ফিরিয়ে আনা হয়। প্রসেসটি eax এর নতুন মানের সাথে এই বিশেষ ইন্সট্রাকশন থেকে এক্সিকিউট হতে থাকে যা সিস্টেম কল থেকে প্রত্যাবর্তন করে। ধন্যবাদ।